

รายวิชา ระบบฐานข้อมูลขั้นสูง

## รายงาน Lab สัปดาห์ที่ 2

Transaction, Concurrency & Recovery

หัวข้อ 2.5 แบบฝึกหัดปฏิบัติ (Lab 1-4) และ 2.6 Lab Assignment (งาน A-F)

จัดทำโดย

**นายจิรภัทร จันทรมล**

รหัสนักศึกษา 664245026 หมู่เรียน 66/43

มหาวิทยาลัยราชภัฏนครปฐม  
ภาคเรียนที่ 1 ปีการศึกษา 2569

## 0. เตรียมสภาพแวดล้อม

### สภาพแวดล้อมที่ใช้ทดลอง

| รายการ                                       | ค่าที่ใช้                                                                         |
|----------------------------------------------|-----------------------------------------------------------------------------------|
| MySQL Server                                 | 8.0.46 (InnoDB) ติดตั้งในเครื่องที่ใช้ทดลอง                                       |
| Client                                       | Python 3 + mysql-connector-python ผ่าน Jupyter Notebook (เปิดบน Google Colab ได้) |
| Authentication                               | caching_sha2_password (ค่า default ไม่ได้ใช้ mysql_native_password)               |
| Isolation default                            | REPEATABLE READ                                                                   |
| innodb_flush_log_at_trx_commit / sync_binlog | 1 / 1                                                                             |

หมายเหตุ: เอกสารกำหนด MySQL 8.4 แต่เครื่องที่ผมใช้ทดลองติดตั้งได้แค่ 8.0.46 ซึ่งมีตัวแปรและพฤติกรรมของ InnoDB ที่ใช้ใน Lab นี้ครบ (เช่น innodb\_redo\_log\_capacity มีตั้งแต่ 8.0.30) ถ้ารันบน server 8.4 ของรายวิชา ตัวเลขเวลาและจำนวน deadlock อาจไม่เท่ากัน เพราะผล concurrency ขึ้นกับจังหวะของ thread ในแต่ละครั้ง

password ไม่ได้เขียนไว้ใน notebook แต่อ่านจาก environment variable MYSQL\_PASSWORD หรือ getpass() และทุกฟังก์ชันปิด cursor/connection ใน finally

```
import os, getpass, time, random, threading, statistics, concurrent.futures
import mysql.connector
from mysql.connector import Error
import pandas as pd

# ไม่ได้ password จริงใน notebook: อ่านจาก environment variable หรือถามตอนรัน
SERVER_CONFIG = {
    'host': os.environ.get('MYSQL_HOST', '127.0.0.1'),
    'port': int(os.environ.get('MYSQL_PORT', '3306')),
    'user': os.environ.get('MYSQL_USER', 'labuser'),
    'password': os.environ.get('MYSQL_PASSWORD') or getpass.getpass('MySQL password: '),
    'connection_timeout': 10,
}

conn = mysql.connector.connect(**SERVER_CONFIG)
cur = conn.cursor()
cur.execute("CREATE DATABASE IF NOT EXISTS fake_news_db CHARACTER SET utf8mb4")
cur.close(); conn.close()

DB_CONFIG = {**SERVER_CONFIG, 'database': 'fake_news_db', 'autocommit': False}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

def query(sql, params=None):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute(sql, params or ())
        rows = cur.fetchall()
        cols = [d[0] for d in cur.description]
        conn.commit()
        return pd.DataFrame(rows, columns=cols)
    finally:
        cur.close(); conn.close()

print('Connector/Python', mysql.connector.__version__)
query("SELECT VERSION() AS mysql_version, @@transaction_isolation AS default_isolation, "
      "@@innodb_flush_log_at_trx_commit AS flush_at_commit, @@innodb_lock_wait_timeout AS lock_wait_timeout")
```

## Output

Connector/Python 26.7.0

|   | mysql_version           | default_isolation | flush_at_commit | lock_wait_timeout |
|---|-------------------------|-------------------|-----------------|-------------------|
| 0 | 8.0.46-0ubuntu0.24.04.4 | REPEATABLE-READ   | 1               | 50                |

```
def setup_database():
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("""
        CREATE TABLE IF NOT EXISTS users (
            user_id VARCHAR(20) PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100) UNIQUE,
            followers INT NOT NULL DEFAULT 0,
            post_count INT NOT NULL DEFAULT 0,
            is_verified BOOLEAN NOT NULL DEFAULT FALSE,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            CHECK (followers >= 0), CHECK (post_count >= 0)
        ) ENGINE=InnoDB""")
        cur.execute("""
        CREATE TABLE IF NOT EXISTS posts (
            post_id VARCHAR(20) PRIMARY KEY,
            author_id VARCHAR(20) NOT NULL,
            content TEXT NOT NULL,
            share_count INT NOT NULL DEFAULT 0,
            report_count INT NOT NULL DEFAULT 0,
            is_flagged BOOLEAN NOT NULL DEFAULT FALSE,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            CONSTRAINT fk_posts_author FOREIGN KEY (author_id)
                REFERENCES users(user_id) ON DELETE CASCADE,
            CHECK (share_count >= 0), CHECK (report_count >= 0)
        ) ENGINE=InnoDB""")
        cur.execute("""
        CREATE TABLE IF NOT EXISTS shares (
            share_id BIGINT AUTO_INCREMENT PRIMARY KEY,
            post_id VARCHAR(20) NOT NULL,
            user_id VARCHAR(20) NOT NULL,
            shared_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            CONSTRAINT fk_shares_post FOREIGN KEY(post_id) REFERENCES posts(post_id) ON DELETE CASCADE,
            CONSTRAINT fk_shares_user FOREIGN KEY(user_id) REFERENCES users(user_id) ON DELETE CASCADE,
            CONSTRAINT uq_share UNIQUE(post_id, user_id)
        ) ENGINE=InnoDB""")
        # ตารางเพิ่มสำหรับ Lab 2
        cur.execute("""
        CREATE TABLE IF NOT EXISTS reports (
            report_id BIGINT AUTO_INCREMENT PRIMARY KEY,
            post_id VARCHAR(20) NOT NULL,
            user_id VARCHAR(20) NOT NULL,
            reason VARCHAR(255) NOT NULL,
            reported_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            CONSTRAINT fk_reports_post FOREIGN KEY(post_id) REFERENCES posts(post_id) ON DELETE CASCADE,
            CONSTRAINT fk_reports_user FOREIGN KEY(user_id) REFERENCES users(user_id) ON DELETE CASCADE,
            CONSTRAINT uq_report UNIQUE(post_id, user_id)
        ) ENGINE=InnoDB""")
        conn.commit()
    except Error:
        conn.rollback(); raise
    finally:
        cur.close(); conn.close()

def reset_data(n_bots=0):
    """ล้างข้อมูลแล้วใส่ข้อมูลตัวอย่างใหม่ (รันซ้ำได้) n_bots = จำนวน user ปลอมสำหรับทดสอบ concurrency"""
    conn = get_connection(); cur = conn.cursor()
    try:
        for t in ('reports', 'shares', 'posts', 'users'):
            cur.execute(f"DELETE FROM {t}")
        users = [(('U001', 'Alice', 'alice@test.com', 1500, 0, True),
                  ('U002', 'Bob', 'bob@test.com', 300, 0, False),
                  ('U003', 'Charlie', 'charlie@test.com', 800, 0, True),
                  ('U004', 'David', 'david@test.com', 120, 0, False))]
        users += [(f'B{i:03d}', f'Bot{i}', None, 0, 0, False) for i in range(1, n_bots+1)]
        cur.executemany("""INSERT INTO users(user_id,name,email,followers,post_count,is_verified)
            VALUES (%s,%s,%s,%s,%s,%s)""", users)
        posts = [(('P001', 'U001', 'Breaking: New AI model released!', 0, 0, False),
                  ('P002', 'U002', 'Is this AI news fake?', 0, 0, False),
                  ('P003', 'U003', 'Government announces new policy', 0, 0, False))]
```

```

cur.executemany("""INSERT INTO posts(post_id,author_id,content,share_count,report_count,is_flagged)
VALUES (%s,%s,%s,%s,%s,%s)""", posts)
conn.commit()
except Error:
    conn.rollback(); raise
finally:
    cur.close(); conn.close()

setup_database()
reset_data()
query("SELECT post_id, author_id, content, share_count, report_count, is_flagged FROM posts")

```

#### Output

|   | post_id | author_id | content                          | share_count | report_count | is_flagged |
|---|---------|-----------|----------------------------------|-------------|--------------|------------|
| 0 | P001    | U001      | Breaking: New AI model released! | 0           | 0            | 0          |
| 1 | P002    | U002      | Is this AI news fake?            | 0           | 0            | 0          |
| 2 | P003    | U003      | Government announces new policy  | 0           | 0            | 0          |

ใช้ schema ตามหัวข้อ 2.1.4 และเพิ่มตาราง reports สำหรับ Lab 2 ฟังก์ชัน reset\_data() ใช้ล้างข้อมูลก่อนทุกการทดลองเพื่อให้รันซ้ำได้

# ส่วนที่ 1: แบบฝึกหัดปฏิบัติ (หัวข้อ 2.5)

## 2.5 Lab 1: วิเคราะห์ Dirty Read

```
def dirty_read_test(level):
    reset_data()
    c1, c2 = get_connection(), get_connection()
    cur1, cur2 = c1.cursor(), c2.cursor()
    result = {'isolation (T2)': level}
    try:
        # T1: แก้วแต่ยังไม่ commit
        cur1.execute("START TRANSACTION")
        cur1.execute("UPDATE posts SET share_count = 777 WHERE post_id = 'P001'")

        # T2: ตั้ง isolation แล้วอ่านค่า
        cur2.execute(f"SET SESSION TRANSACTION ISOLATION LEVEL {level}")
        cur2.execute("SELECT share_count FROM posts WHERE post_id = 'P001'")
        result['T2 อ่านระหว่าง T1 ยังไม่ commit'] = cur2.fetchone()[0]
        c2.commit()

        # T1: ROLLBACK แล้ว T2 อ่านอีกครั้ง
        c1.rollback()
        cur2.execute("SELECT share_count FROM posts WHERE post_id = 'P001'")
        result['T2 อ่านหลัง T1 rollback'] = cur2.fetchone()[0]
        c2.commit()
    finally:
        cur1.close(); cur2.close(); c1.close(); c2.close()
    return result

pd.DataFrame([dirty_read_test('READ UNCOMMITTED'), dirty_read_test('READ COMMITTED')])
```

### Output

|   | isolation (T2)   | T2 อ่านระหว่าง T1 ยังไม่ commit | T2 อ่านหลัง T1 rollback |
|---|------------------|---------------------------------|-------------------------|
| 0 | READ UNCOMMITTED | 777                             | 0                       |
| 1 | READ COMMITTED   | 0                               | 0                       |

### ผลที่ได้

- READ UNCOMMITTED: T2 อ่านได้ค่า 777 ทั้งที่ T1 ยังไม่ commit (เกิด dirty read) และพอ T1 rollback ค่าก็กลับเป็น 0 แปลว่าค่า 777 ที่ T2 เห็นไม่เคยมีอยู่จริงในฐานข้อมูลเลย
- READ COMMITTED: T2 อ่านได้ 0 ซึ่งเป็นค่าล่าสุดที่ commit แล้ว ไม่เห็นค่าที่ยังไม่ commit

**ความเสี่ยงต่อ business logic:** ถ้าระบบเอาค่าที่อ่านแบบ dirty ไปตัดสินใจ เช่น เห็น share\_count = 777 แล้วดันโพสต์ขึ้น trending หรือ flag ว่าเป็นข่าวไวรัล การตัดสินใจนั้นจะอิงข้อมูลที่ถูก rollback ไปแล้ว ระบบจึงไม่ควรใช้ READ UNCOMMITTED กับงานที่ต้องการความถูกต้อง

## 2.5 Lab 2: Atomic Transaction สำหรับ Report Fake News

```
FLAG_THRESHOLD = 3
ER_DUP_ENTRY, ER_NO_REFERENCED_ROW = 1062, 1452

def report_fake_news(user_id, post_id, reason, threshold=FLAG_THRESHOLD):
    conn = get_connection(); cur = conn.cursor()
    try:
        # ล็อกแถวของโพสต์ก่อน (X lock) เพื่อให้ report ของโพสต์เดียวกันทำทีละ transaction
        cur.execute("SELECT report_count FROM posts WHERE post_id = %s FOR UPDATE", (post_id,))
        if cur.fetchone() is None:
            raise ValueError(f"ไม่พบโพสต์ {post_id}")

        # 1) บันทึกรายงาน (UNIQUE(post_id, user_id) กัน report ซ้ำ)
```

```

cur.execute("INSERT INTO reports(post_id, user_id, reason) VALUES (%s, %s, %s)",
            (post_id, user_id, reason))
# 2) เพิ่มตัวนับแบบ atomic UPDATE
cur.execute("UPDATE posts SET report_count = report_count + 1 WHERE post_id = %s", (post_id,))
# 3) ถึง threshold แล้ว flag
cur.execute("UPDATE posts SET is_flagged = TRUE
            WHERE post_id = %s AND report_count >= %s", (post_id, threshold))

conn.commit()
return 'committed'
except Error as e:
    conn.rollback()
    # rollback ทั้ง business transaction
    if e.errno == ER_DUP_ENTRY:
        return 'rollback: report ซ้ำ (1062)'
    if e.errno == ER_NO_REFERENCED_ROW:
        return 'rollback: FK error (1452)'
    raise
except ValueError as e:
    conn.rollback()
    return f'rollback: {e}'
finally:
    cur.close(); conn.close()

def post_state(post_id):
    return query("""SELECT p.post_id, p.report_count, p.is_flagged,
        (SELECT COUNT(*) FROM reports r WHERE r.post_id = p.post_id) AS rows_in_reports
        FROM posts p WHERE p.post_id = %s""", (post_id,))

reset_data()
steps = [(('U002', 'P001', 'ข่าวไม่มีแหล่งที่มา'),
          ('U002', 'P001', 'กดซ้ำ'),
          ('U999', 'P001', 'user ไม่มีอยู่จริง'),
          ('U003', 'P999', 'โพสต์ไม่มีอยู่จริง'),
          ('U003', 'P001', 'เนื้อหาบิดเบือน'),
          ('U004', 'P001', 'ข่าวปลอม'))]
log = []
for u, p, r in steps:
    status = report_fake_news(u, p, r)
    s = post_state('P001').iloc[0]
    log.append({'user': u, 'post': p, 'ผลลัพธ์': status,
               'P001.report_count': s.report_count, 'rows_in_reports': s.rows_in_reports,
               'is_flagged': bool(s.is_flagged)})
pd.DataFrame(log)

```

## Output

|   | user | post | ผลลัพธ์                     | P001.report_count | rows_in_reports | is_flagged |
|---|------|------|-----------------------------|-------------------|-----------------|------------|
| 0 | U002 | P001 | committed                   | 1                 | 1               | False      |
| 1 | U002 | P001 | rollback: report ซ้ำ (1062) | 1                 | 1               | False      |
| 2 | U999 | P001 | rollback: FK error (1452)   | 1                 | 1               | False      |
| 3 | U003 | P999 | rollback: ไม่มีพบโพสต์ P999 | 1                 | 1               | False      |
| 4 | U003 | P001 | committed                   | 2                 | 2               | False      |
| 5 | U004 | P001 | committed                   | 3                 | 3               | True       |

ลำดับการทำงานใน transaction (ทั้งหมดอยู่ใน transaction เดียว)

1. SELECT ... FOR UPDATE ล็อกแถวของโพสต์ไว้ก่อน และเช็คว่ามีอยู่จริง
2. INSERT ลง reports (มี UNIQUE(post\_id, user\_id) กัน report ซ้ำ)
3. UPDATE posts SET report\_count = report\_count + 1
4. ถ้า report\_count >= 3 ให้ is\_flagged = TRUE

ผลที่ได้: กรณี report ซ้ำ (1062), user ไม่มีจริง (1452) และโพสต์ไม่มีจริง ทุกกรณี rollback ทั้งชุด สังเกตได้จาก report\_count กับจำนวนแถวใน reports ไม่เปลี่ยนแปลง และเมื่อ report ครบ 3 คน (U002, U003, U004) โพสต์ถูก flag ทันที

ทดสอบ report พร้อมกันหลาย thread เปรียบเทียบเวอร์ชันที่ INSERT ก่อนแล้วค่อย UPDATE (ไม่ล็อกแถวโพสต์ก่อน) กับเวอร์ชันข้างบน

```

def report_fake_news_naive(user_id, post_id, reason, threshold=FLAG_THRESHOLD):
    """เวอร์ชันแรกๆที่เขียน: INSERT ก่อน แล้วค่อย UPDATE posts (ไม่มี SELECT ... FOR UPDATE)"""
    conn = get_connection(); cur = conn.cursor()

```

```

try:
    cur.execute("INSERT INTO reports(post_id, user_id, reason) VALUES (%s, %s, %s)",
                (post_id, user_id, reason))
    time.sleep(0.01)
    cur.execute("UPDATE posts SET report_count = report_count + 1 WHERE post_id = %s", (post_id,))
    cur.execute("UPDATE posts SET is_flagged = TRUE WHERE post_id = %s AND report_count >= %s",
                (post_id, threshold))
    conn.commit()
    return 'committed'
except Error as e:
    conn.rollback()
    return f'error {e.errno}'
finally:
    cur.close(); conn.close()

def concurrent_reports(fn, n=30, rounds=3):
    rows = []
    for r in range(rounds):
        reset_data(n_bots=n)
        users = [f'B{i:03d}' for i in range(1, n+1)] + ['B001', 'B002'] # 2 รายการเป็น report ชั่ว
        with concurrent.futures.ThreadPoolExecutor(max_workers=10) as ex:
            res = list(ex.map(lambda u: fn(u, 'P002', 'bot report'), users))
        s = post_state('P002').iloc[0]
        rows.append({'function': fn.__name__, 'round': r+1, 'requests': len(users),
                    'committed': res.count('committed'),
                    'deadlock (1213)': sum('1213' in x for x in res),
                    'duplicate (1062)': sum('1062' in x for x in res),
                    'report_count': s.report_count, 'rows_in_reports': s.rows_in_reports,
                    'ตรงกัน?': s.report_count == s.rows_in_reports})

    return rows

pd.DataFrame(concurrent_reports(report_fake_news_naive) + concurrent_reports(report_fake_news))

```

## Output

|   | function               | round | requests | committed | deadlock (1213) | duplicate (1062) | report_count | rows_in_reports | ตรงกัน? |
|---|------------------------|-------|----------|-----------|-----------------|------------------|--------------|-----------------|---------|
| 0 | report_fake_news_naive | 1     | 32       | 19        | 11              | 2                | 19           | 19              | True    |
| 1 | report_fake_news_naive | 2     | 32       | 15        | 15              | 2                | 15           | 15              | True    |
| 2 | report_fake_news_naive | 3     | 32       | 18        | 13              | 1                | 18           | 18              | True    |
| 3 | report_fake_news       | 1     | 32       | 30        | 0               | 2                | 30           | 30              | True    |
| 4 | report_fake_news       | 2     | 32       | 30        | 0               | 2                | 30           | 30              | True    |
| 5 | report_fake_news       | 3     | 32       | 30        | 0               | 2                | 30           | 30              | True    |

ผลที่ได้: ส่ง report พร้อมกัน 32 ครั้ง (bot 30 คน + report ชั่ว 2 ครั้ง) รอบละ 3 รอบ

- **report\_fake\_news\_naive** (INSERT ก่อนแล้วค่อย UPDATE) เกิด **deadlock 11-15 ครั้ง** ต่อรอบ สาเหตุคือตอน INSERT ลง reports InnoDB ต้องเช็ค foreign key จึงตั้ง **shared lock** ที่แถว p002 ในตาราง posts พอหลาย transaction ถือ shared lock แถวเดียวกันอยู่ แล้วทุกตัวจะ UPDATE แถวนั้น (ต้องใช้ exclusive lock) ก็เลยรอกันเอง InnoDB จึงเลือก rollback บางตัวทิ้ง
- ข้อมูลยังตรงกัน (report\_count = จำนวนแถวใน reports) เพราะ transaction ที่โดน deadlock ถูก rollback ทั้งชุด แต่ **report** ของคนเหล่านั้นหายไป ถ้าไม่ retry
- **report\_fake\_news** (ล็อกแถวโพสต์ด้วย SELECT ... FOR UPDATE ก่อน) **ไม่เกิด deadlock** เลย report สำเร็จครบ 30 และ report ชั่วถูกปฏิเสธ 2 ครั้งตามที่ควร

**สรุปวิธีที่ควรใช้:** ตัวนับให้ใช้ atomic UPDATE (report\_count = report\_count + 1) ไม่อ่านค่ามาบวกใน Python และถ้า transaction ต้องแตะแถวเดียวกันหลายครั้งให้ขอ exclusive lock ตั้งแต่แรก (SELECT ... FOR UPDATE) เพื่อไม่ให้เกิดการอัปเดตจาก shared lock เป็น exclusive lock นอกจากนี้ควรมี retry ไว้รองรับ deadlock ที่อาจเกิดจากจุดอื่นด้วย

## 2.5 Lab 3: Benchmark Autocommit vs Explicit Transaction

```
def global_status(name):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("SHOW GLOBAL STATUS LIKE %s", (name,))
        return int(cur.fetchone()[1])
    finally:
        cur.close(); conn.close()

def bench_setup():
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("""CREATE TABLE IF NOT EXISTS bench_rows (
            id INT AUTO_INCREMENT PRIMARY KEY, payload VARCHAR(100)) ENGINE=InnoDB""")
        cur.execute("TRUNCATE TABLE bench_rows")
        conn.commit()
    finally:
        cur.close(); conn.close()

def run_case(case, n):
    bench_setup()
    conn = get_connection(); cur = conn.cursor()
    conn.autocommit = (case == 'A') # A: autocommit ทุก statement, B: commit ครั้งเดียว
    fsync0 = global_status('Innodb_os_log_fsyncs')
    t0 = time.perf_counter()
    try:
        for i in range(n):
            cur.execute("INSERT INTO bench_rows(payload) VALUES (%s)", (f'row {i}',))
            if case == 'B':
                conn.commit()
    except Error:
        if case == 'B': conn.rollback()
        raise
    finally:
        elapsed = time.perf_counter() - t0
        cur.close(); conn.close()
    return elapsed, global_status('Innodb_os_log_fsyncs') - fsync0

N, ROUNDS = 1000, 5
run_case('A', 100); run_case('B', 100) # warm up
runs = []
for r in range(ROUNDS):
    for case in ('A', 'B'):
        sec, fsyncs = run_case(case, N)
        runs.append({'case': case, 'round': r+1, 'seconds': sec, 'redo_fsyncs': fsyncs})
runs = pd.DataFrame(runs)
summary = runs.groupby('case').agg(median_s=('seconds', 'median'), mean_s=('seconds', 'mean'),
                                   min_s=('seconds', 'min'), max_s=('seconds', 'max'),
                                   median_redo_fsyncs=('redo_fsyncs', 'median')).round(4)
summary['rows_per_sec'] = (N / summary['median_s']).round(0)
summary.index = ['A: autocommit=True', 'B: explicit transaction']
print(runs.pivot(index='round', columns='case', values='seconds').round(4))
summary
```

### Output

| case  | A      | B      |
|-------|--------|--------|
| round |        |        |
| 1     | 1.0523 | 0.2501 |
| 2     | 1.0164 | 0.2511 |
| 3     | 0.9377 | 0.3448 |
| 4     | 0.9294 | 0.2359 |
| 5     | 0.9586 | 0.3031 |

|                         | median_s | mean_s | min_s  | max_s  | median_redo_fsyncs | rows_per_sec |
|-------------------------|----------|--------|--------|--------|--------------------|--------------|
| A: autocommit=True      | 0.9586   | 0.9789 | 0.9294 | 1.0523 | 1139.0             | 1043.0       |
| B: explicit transaction | 0.2511   | 0.2770 | 0.2359 | 0.3448 | 234.0              | 3982.0       |

ผลที่ได้ (INSERT 1,000 แถว, warm up 1 รอบ, วัด 5 รอบ)

- Case A (autocommit ทุก statement) ใช้เวลา median 0.96 วินาที (~1,040 แถว/วินาที)



- Case B (explicit transaction, commit ครั้งเดียว) ใช้เวลา median 0.25 วินาที (~3,980 แถว/วินาที) เร็วกว่าประมาณ 3.8 เท่า
- จำนวน redo log fsync ของ A อยู่ที่ประมาณ 1,139 ครั้ง (ประมาณ 1 ครั้งต่อ commit) ส่วน B มีประมาณ 234 ครั้ง ซึ่งน่าจะมาจาก background thread ของ InnoDB ที่ทยอย flush redo ระหว่างทาง ไม่ได้มาจากการ commit

**อธิบาย:** เมื่อ `innodb_flush_log_at_trx_commit = 1` ทุก commit ต้องเขียน redo log ลง disk และ fsync ให้เสร็จก่อนตอบกลับ (และเครื่องนี้เปิด binary log กับ `sync_binlog = 1` ด้วย จึงมี fsync ของ binlog อีกชั้น) การ commit ที่ละแถวจึงเสียเวลากับ fsync เป็นส่วนใหญ่ แต่ก็ไม่ควรทำ transaction ใหญ่เกินไป เพราะ transaction ที่ยาวจะถือ lock นาน, สร้าง undo log เยอะ (purge ช้าและ MVCC ต้องเก็บ version เก่าไว้นาน) และถ้า rollback หรือเครื่องล่มกลางทางก็ต้องใช้เวลา rollback นาน แนวทางที่เหมาะสมคือ commit เป็น batch ขนาดพอดี เช่น ครั้งละหลายร้อยถึงหลักพันแถว

## 2.5 Lab 4: Deadlock Detection และ Retry

```
def deadlock_demo():
    reset_data()
    barrier = threading.Barrier(2, timeout=5)  # ให้ทั้งสองฝั่งถือ lock แรกไว้ก่อน
    result = {}
    def worker(name, first, second):
        conn = get_connection(); cur = conn.cursor()
        try:
            cur.execute("START TRANSACTION")
            cur.execute("UPDATE users SET followers = followers + 1 WHERE user_id = %s", (first,))
            barrier.wait()
            cur.execute("UPDATE users SET followers = followers + 1 WHERE user_id = %s", (second,))
            conn.commit()
            result[name] = 'committed'
        except Error as e:
            conn.rollback()
            result[name] = f'error {e.errno}: {e.msg}'
        finally:
            cur.close(); conn.close()
    t1 = threading.Thread(target=worker, args=('T1 (U001 -> U002)', 'U001', 'U002'))
    t2 = threading.Thread(target=worker, args=('T2 (U002 -> U001)', 'U002', 'U001'))
    t1.start(); t2.start(); t1.join(); t2.join()
    return result

for k, v in deadlock_demo().items():
    print(f'{k:20s} {v}')
print(query("SELECT user_id, followers FROM users WHERE user_id IN ('U001','U002)').to_string(index=False))
```

### Output

```
T2 (U002 -> U001)    error 1213: Deadlock found when trying to get lock; try restarting transaction
T1 (U001 -> U002)    committed
user_id followers
U001          1501
U002          301
```

T1 ล็อกแถว u001 และ T2 ล็อกแถว u002 ก่อน (ใช้ `threading.Barrier` ให้ทั้งคู่ถือ lock แรกพร้อมกัน) จากนั้นต่างฝ่ายต่างขอแถวที่อีกฝ่ายถืออยู่ InnoDB ตรวจเจอ deadlock ทันทีและ rollback ฝั่งหนึ่ง (error 1213) อีกฝั่ง commit ได้ตามปกติ

ผลลัพธ์ u001 = 1501, u002 = 301 คือได้แค่ +1 จาก transaction ที่ commit เท่านั้น ส่วน transaction ที่ถูกเลือกเป็น victim ถูก rollback **ทั้งชุด** ไม่มีการค้างครั้งเดียว

```
def latest_deadlock_section():
    conn = get_connection(); cur = conn.cursor()
    try:
```

```

cur.execute("SHOW ENGINE INNODB STATUS")
status = cur.fetchone()[2]
finally:
    cur.close(); conn.close()
start = status.find('LATEST DETECTED DEADLOCK')
end = status.find('TRANSACTIONS', start + 30)
keep = ('***', 'TRANSACTION', 'UPDATE', 'RECORD LOCKS', 'WE ROLL BACK', 'LATEST', 'mysql tables',
        'LOCK WAIT', 'lock struct')
lines = [l for l in status[start:end].splitlines() if any(l.strip().startswith(k) for k in keep)]
return '\n'.join(l.replace(' space id 2 page no 4 n bits 104', '') for l in lines)

print(latest_deadlock_section())

```

### Output

```

LATEST DETECTED DEADLOCK
*** (1) TRANSACTION:
TRANSACTION 23788, ACTIVE 0 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
UPDATE users SET followers = followers + 1 WHERE user_id = 'U002'
*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 2 page no 4 n bits 112 index PRIMARY of table `fake_news_db`.`users` trx id 23788
lock_mode X locks rec but not gap
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 2 page no 4 n bits 112 index PRIMARY of table `fake_news_db`.`users` trx id 23788
lock_mode X locks rec but not gap waiting
*** (2) TRANSACTION:
TRANSACTION 23787, ACTIVE 0 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
UPDATE users SET followers = followers + 1 WHERE user_id = 'U001'
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 2 page no 4 n bits 112 index PRIMARY of table `fake_news_db`.`users` trx id 23787
lock_mode X locks rec but not gap
*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 2 page no 4 n bits 112 index PRIMARY of table `fake_news_db`.`users` trx id 23787
lock_mode X locks rec but not gap waiting
*** WE ROLL BACK TRANSACTION (2)

```

### อ่านจาก SHOW ENGINE INNODB STATUS (ส่วน LATEST DETECTED DEADLOCK)

- transaction (1) **HOLDS X lock** ที่ record หนึ่งของ index PRIMARY ของ users และ **WAITING** lock ที่อีก record หนึ่ง
- transaction (2) ก็ถือ lock ของอีก record แล้วรอ record ที่ (1) ถืออยู่ เกิดเป็นวงรอ (cycle) ใน wait-for graph
- lock\_mode X locks rec but not gap หมายถึง exclusive lock เฉพาะ record ไม่รวม gap เพราะเป็นการ UPDATE ด้วย primary key ตรงตัว
- บรรทัดสุดท้าย **WE ROLL BACK TRANSACTION (2)** คือ InnoDB เลือก transaction (2) เป็น victim

DEADLOCK, LOCK\_WAIT\_TIMEOUT = 1213, 1205

```

def run_with_retry(fn, max_retries=5, log=None, name=''):
    for attempt in range(max_retries):
        try:
            out = fn()
            if log is not None: log.append({'txn': name, 'attempt': attempt+1, 'result': 'committed',
            'wait_s': 0})
            return out
        except Error as e:
            if e.errno not in (DEADLOCK, LOCK_WAIT_TIMEOUT) or attempt == max_retries - 1:
                raise
            wait = 0.1 * (2 ** attempt) + random.uniform(0, 0.05) # exponential backoff + jitter
            if log is not None: log.append({'txn': name, 'attempt': attempt+1, 'result': f'error {e.errno} ->
            retry', 'wait_s': round(wait, 3)})
            time.sleep(wait)

def transfer_followers(from_user, to_user, amount):
    conn = get_connection(); cur = conn.cursor()

```

```

try:
    cur.execute("UPDATE users SET followers = followers - %s WHERE user_id = %s", (amount, from_user))
    time.sleep(0.2)
    cur.execute("UPDATE users SET followers = followers + %s WHERE user_id = %s", (amount, to_user))
    conn.commit()
except Error:
    conn.rollback(); raise
finally:
    cur.close(); conn.close()

reset_data()
before = query("SELECT SUM(followers) AS total FROM users WHERE user_id IN ('U001','U002)').total[0]
log = []
threads = [threading.Thread(target=run_with_retry, kwargs=dict(fn=lambda: transfer_followers('U001', 'U002'),
10), log=log, name='T1 U001->U002')),
            threading.Thread(target=run_with_retry, kwargs=dict(fn=lambda: transfer_followers('U002', 'U001'),
25), log=log, name='T2 U002->U001'))]
for t in threads: t.start()
for t in threads: t.join()
print(pd.DataFrame(log).to_string(index=False))
after = query("SELECT user_id, followers FROM users WHERE user_id IN ('U001','U002)")
print(after.to_string(index=False))
print('ผลรวม followers ก่อน =', before, ' หลัง =', after.followers.sum())

```

#### Output

| txn           | attempt | result              | wait_s |
|---------------|---------|---------------------|--------|
| T2 U002->U001 | 1       | error 1213 -> retry | 0.103  |
| T1 U001->U002 | 1       | committed           | 0.000  |
| T2 U002->U001 | 2       | committed           | 0.000  |

  

| user_id | followers |
|---------|-----------|
| U001    | 1515      |
| U002    | 285       |

ผลรวม followers ก่อน = 1800 หลัง = 1800

**Retry ทั้ง transaction ด้วย exponential backoff + jitter:** T2 โดน deadlock ในรอบแรก รอประมาณ 0.1 วินาทีแล้วรันใหม่ทั้ง transaction จึงสำเร็จในรอบที่ 2 ผลรวม followers ก่อนและหลังเท่ากัน (1,800) และค่าสุดท้ายถูกต้อง (U001 = 1500 - 10 + 25 = 1515, U002 = 300 + 10 - 25 = 285)

ที่ต้อง retry ทั้ง transaction (ไม่ใช่แค่ statement ที่ error) เพราะเมื่อเกิด deadlock InnoDB rollback victim ทั้ง transaction ไปแล้ว ส่วน jitter ช่วยให้สอง transaction ไม่กลับมาชนกันในจังหวะเดิมอีก

```

def transfer_ordered(from_user, to_user, amount):
    """ล็อกแถวตามลำดับ user_id เสมอ ไม่ว่าจะโอนจากใครไปใคร"""
    conn = get_connection(); cur = conn.cursor()
    try:
        a, b = sorted([from_user, to_user])
        cur.execute("SELECT user_id FROM users WHERE user_id IN (%s, %s) ORDER BY user_id FOR UPDATE", (a,
b))
        cur.fetchall()
        cur.execute("UPDATE users SET followers = followers - %s WHERE user_id = %s", (amount, from_user))
        time.sleep(0.02)
        cur.execute("UPDATE users SET followers = followers + %s WHERE user_id = %s", (amount, to_user))
        conn.commit(); return 'committed'
    except Error as e:
        conn.rollback(); return f'error {e.errno}'
    finally:
        cur.close(); conn.close()

def transfer_unordered(from_user, to_user, amount):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("UPDATE users SET followers = followers - %s WHERE user_id = %s", (amount, from_user))
        time.sleep(0.02)
        cur.execute("UPDATE users SET followers = followers + %s WHERE user_id = %s", (amount, to_user))
        conn.commit(); return 'committed'
    except Error as e:
        conn.rollback(); return f'error {e.errno}'
    finally:
        cur.close(); conn.close()

```

```

rows = []
for fn in (transfer_unordered, transfer_ordered):
    reset_data()
    jobs = [('U001', 'U002', 1), ('U002', 'U001', 1)] * 20 # 40 transactions สลับทิศ
    with concurrent.futures.ThreadPoolExecutor(max_workers=8) as ex:
        res = list(ex.map(lambda j: fn(*j), jobs))
    rows.append({'function': fn.__name__, 'transactions': len(res),
                'committed': res.count('committed'), 'deadlock (1213)': res.count('error 1213')})
pd.DataFrame(rows)

```

#### Output

|   | function           | transactions | committed | deadlock (1213) |
|---|--------------------|--------------|-----------|-----------------|
| 0 | transfer_unordered | 40           | 30        | 10              |
| 1 | transfer_ordered   | 40           | 40        | 0               |

ป้องกันตั้งแต่ต้นด้วยการล็อกตามลำดับเดียวกัน: ส่ง transaction โอน followers สลับทิศไปมา 40 ครั้ง

- ไม่เรียงลำดับ: เกิด deadlock 10 ครั้งจาก 40
- ล็อกแถวตาม user\_id จากน้อยไปมากเสมอ (SELECT ... ORDER BY user\_id FOR UPDATE): ไม่เกิด deadlock เลย เพราะไม่มีทางเกิดวงรอได้

ในระบบจริงควรใช้ทั้งสองอย่างคู่กัน คือออกแบบลำดับการล็อกให้เหมือนกันทุกที่ และยังมี retry ไว้เพื่อ deadlock จากกรณีที่ไม่คาดไม่ถึง

## ส่วนที่ 2: Lab Assignment (หัวข้อ 2.6)

### A. ACID Transaction: share\_post

```
class SimulatedCrash(Exception):
    pass

def share_post(user_id, post_id, fail_after_update=False):
    """business operation: บันทึกการแชร์ + เพิ่ม share_count ต้องสำเร็จทั้งคู่หรือไม่เกิดเลย"""
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("INSERT INTO shares(post_id, user_id) VALUES (%s, %s)", (post_id, user_id))
        cur.execute("UPDATE posts SET share_count = share_count + 1 WHERE post_id = %s", (post_id,))
        if cur.rowcount != 1:
            raise ValueError(f'Unknown post_id: {post_id}')
        if fail_after_update:
            raise SimulatedCrash('จำลอง error หลัง UPDATE ก่อน COMMIT')
        conn.commit()
        return 'committed'
    except Exception as e:
        conn.rollback()
        return f'rollback ({type(e).__name__}: {str(e.errno) if isinstance(e, Error) else ""})'
    finally:
        cur.close(); conn.close()

def share_state():
    return query("""SELECT (SELECT share_count FROM posts WHERE post_id='P001') AS share_count,
        (SELECT COUNT(*) FROM shares WHERE post_id='P001') AS rows_in_shares""").iloc[0]

reset_data()
cases = [('แชร์ปกติ', dict(user_id='U002', post_id='P001')),
        ('แชร์ซ้ำ (UNIQUE)', dict(user_id='U002', post_id='P001')),
        ('user ไม่มีจริง (FK)', dict(user_id='U999', post_id='P001')),
        ('error หลัง UPDATE', dict(user_id='U003', post_id='P001', fail_after_update=True)),
        ('แชร์ปกติ', dict(user_id='U004', post_id='P001'))]

out = []
for label, kw in cases:
    res = share_post(**kw); s = share_state()
    out.append({'กรณี': label, 'user': kw['user_id'], 'ผลลัพธ์': res,
        'share_count': s.share_count, 'rows_in_shares': s.rows_in_shares})
pd.DataFrame(out)
```

#### Output

|   | กรณี                | user | ผลลัพธ์                         | share_count | rows_in_shares |
|---|---------------------|------|---------------------------------|-------------|----------------|
| 0 | แชร์ปกติ            | U002 | committed                       | 1           | 1              |
| 1 | แชร์ซ้ำ (UNIQUE)    | U002 | rollback (IntegrityError: 1062) | 1           | 1              |
| 2 | user ไม่มีจริง (FK) | U999 | rollback (IntegrityError: 1452) | 1           | 1              |
| 3 | error หลัง UPDATE   | U003 | rollback (SimulatedCrash)       | 1           | 1              |
| 4 | แชร์ปกติ            | U004 | committed                       | 2           | 2              |

share\_post() มี 2 statement ที่ต้องเกิดพร้อมกัน คือ INSERT ลง shares และ UPDATE share\_count ผลทดสอบ 5 กรณี:

| กรณี                          | ผล             | หลักฐานว่า rollback ทั้งชุด                           |
|-------------------------------|----------------|-------------------------------------------------------|
| แชร์ซ้ำ                       | 1062 duplicate | share_count ยังเป็น 1 และ shares ยังมี 1 แถว          |
| user ไม่มีจริง                | 1452 FK        | ค่าไม่เปลี่ยนแปลง                                     |
| error หลัง UPDATE ก่อน COMMIT | SimulatedCrash | UPDATE ที่ทำไปแล้วถูกยกเลิก share_count ไม่ขึ้นเป็น 2 |

ทุกครั้งที่ commit สำเร็จ share\_count เท่ากับจำนวนแถวใน shares เสมอ ซึ่งเป็น invariant ของระบบ (Consistency)

ถ้าไม่ rollback เอง: error ของ statement ที่สองไม่ได้ทำให้ statement แรกหายไป

```
reset_data()
conn = get_connection(); cur = conn.cursor()
try:
    cur.execute("INSERT INTO shares(post_id, user_id) VALUES ('P001', 'U002')"); conn.commit() # มีอยู่แล้ว 1
    # แล้ว

    cur.execute("START TRANSACTION")
    cur.execute("UPDATE posts SET share_count = share_count + 1 WHERE post_id = 'P001'") # statement 1
    # สำเร็จ

    try:
        cur.execute("INSERT INTO shares(post_id, user_id) VALUES ('P001', 'U002')") # statement 2 ชั่ว
    except Error as e:
        print('statement 2 error:', e.errno, e.msg)
        cur.execute("SELECT share_count FROM posts WHERE post_id = 'P001'")
        print('share_count ภายใน transaction เดิมหลัง error =', cur.fetchone()[0])
        conn.rollback()
        print('หลัง application สั่ง ROLLBACK =', query("SELECT share_count FROM posts WHERE
        post_id='P001'").share_count[0])
    finally:
        cur.close(); conn.close()
```

#### Output

statement 2 error: 1062 Duplicate entry 'P001-U002' for key 'shares.uq\_share'  
share\_count ภายใน transaction เดิมหลัง error = 1  
หลัง application สั่ง ROLLBACK = 0

**ข้อสังเกตสำคัญ:** เมื่อ statement ที่ 2 error ด้วย duplicate key, InnoDB rollback เฉพาะ statement นั้น ส่วน UPDATE ก่อนหน้ายังคงค้างอยู่ใน transaction (share\_count = 1) ถ้าโปรแกรมจับ error แล้วสั่ง commit ต่อเลย จะได้ข้อมูลครั้งเดียว (share\_count เพิ่มขึ้นแต่ไม่มีแถวการแชร์ใหม่) ดังนั้น Atomicity ของ business operation ต้องมาจากการที่ application สั่ง ROLLBACK เองใน exception path ซึ่งตรงกับคำถามอภิปรายข้อ 1

## B. Lost Update

```
def bot_share_unsafe(post_id):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("SELECT share_count FROM posts WHERE post_id = %s", (post_id,))
        current = cur.fetchone()[0]
        time.sleep(0.02) # เพิ่มโอกาส race
        cur.execute("UPDATE posts SET share_count = %s WHERE post_id = %s", (current + 1, post_id))
        conn.commit()
    except Exception:
        conn.rollback(); raise
    finally:
        cur.close(); conn.close()

def bot_share_atomic(post_id):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("UPDATE posts SET share_count = share_count + 1 WHERE post_id = %s", (post_id,))
        conn.commit()
    except Exception:
        conn.rollback(); raise
    finally:
        cur.close(); conn.close()

def bot_share_for_update(post_id):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("SELECT share_count FROM posts WHERE post_id = %s FOR UPDATE", (post_id,))
        current = cur.fetchone()[0]
        time.sleep(0.02)
        cur.execute("UPDATE posts SET share_count = %s WHERE post_id = %s", (current + 1, post_id))
        conn.commit()
    except Exception:
        conn.rollback(); raise
    finally:
        cur.close(); conn.close()
```

```

cur.close(); conn.close()

BOTS, ROUNDS = 10, 5
rows = []
for fn in (bot_share_unsafe, bot_share_atomic, bot_share_for_update):
    results = []
    for r in range(ROUNDS):
        reset_data()
        with concurrent.futures.ThreadPoolExecutor(max_workers=BOTS) as ex:
            list(ex.map(lambda i: fn('P001'), range(BOTS)))
        results.append(int(query("SELECT share_count FROM posts WHERE post_id='P001'").share_count[0]))
    rows.append({'วิธี': fn.__name__, 'คาดหวัง': BOTS, 'ผลแต่ละรอบ': results,
                'lost updates เฉลี่ย': BOTS - statistics.mean(results)})
pd.DataFrame(rows)

```

#### Output

|   | วิธี                 | คาดหวัง | ผลแต่ละรอบ           | lost updates เฉลี่ย |
|---|----------------------|---------|----------------------|---------------------|
| 0 | bot_share_unsafe     | 10      | [2, 4, 2, 2, 4]      | 7.2                 |
| 1 | bot_share_atomic     | 10      | [10, 10, 10, 10, 10] | 0.0                 |
| 2 | bot_share_for_update | 10      | [10, 10, 10, 10, 10] | 0.0                 |

ผลที่ได้ (bot 10 ตัวแชร์พร้อมกัน 5 รอบ คาดว่าต้องได้ 10)

- **unsafe read-modify-write**: ได้แค่ 2-4 ต่อบรอบ เฉลี่ยหายไป 7.2 ครั้ง เพราะหลาย thread อ่านค่าเดิมพร้อมกัน แล้วเขียน `current + 1` ทับกัน (ภายใต้ REPEATABLE READ ตัว SELECT ธรรมดาเป็น consistent read ที่ไม่ล็อกอะไรเลย)
- **atomic UPDATE** (`share_count = share_count + 1`): ได้ 10 ทุกรอบ เพราะการคำนวณเกิดภายใน UPDATE ซึ่งถือ X lock ของแถวนั้นอยู่
- **SELECT ... FOR UPDATE**: ได้ 10 ทุกรอบ เพราะ locking read ทำให้ thread ถัดไปต้องรอนกว่าตัวแรก commit แล้วจึงอ่านค่าล่าสุด

ถ้าแค่บวกตัวเลขให้ใช้ atomic UPDATE จะง่ายและเร็วที่สุด ส่วน FOR UPDATE เหมาะกับกรณีที่ต้องเอาค่าที่อ่านไปตัดสินใจ logic อื่นก่อนเขียน

## C. Isolation Level

```

def reader(level, ready, writer_done, result):
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute(f"SET SESSION TRANSACTION ISOLATION LEVEL {level}")
        cur.execute("START TRANSACTION")
        cur.execute("SELECT share_count FROM posts WHERE post_id = 'P001'")
        result['first read'] = cur.fetchone()[0]
        ready.set()
        writer_done.wait(timeout=8)
        cur.execute("SELECT share_count FROM posts WHERE post_id = 'P001'")
        result['second read'] = cur.fetchone()[0]
        conn.commit()
    finally:
        cur.close(); conn.close()

def writer(ready, writer_done, result):
    ready.wait(timeout=8)
    conn = get_connection(); cur = conn.cursor()
    t0 = time.perf_counter()
    try:
        cur.execute("SET SESSION innodb_lock_wait_timeout = 3")
        cur.execute("UPDATE posts SET share_count = 999 WHERE post_id = 'P001'")
        conn.commit(); result['writer'] = 'committed'
    except Error as e:
        conn.rollback(); result['writer'] = f'error {e.errno}'
    finally:
        result['writer wait (s)'] = round(time.perf_counter() - t0, 2)
        cur.close(); conn.close(); writer_done.set()

def run_isolation_test(level):

```



```

reset_data()
ready, done, result = threading.Event(), threading.Event(), {'level': level}
t1 = threading.Thread(target=reader, args=(level, ready, done, result))
t2 = threading.Thread(target=writer, args=(ready, done, result))
t1.start(); t2.start(); t1.join(); t2.join()
result['ค่าหลังจบ'] = int(query("SELECT share_count FROM posts WHERE post_id='P001'").share_count[0])
return result

pd.DataFrame([run_isolation_test(l) for l in ('READ COMMITTED', 'REPEATABLE READ', 'SERIALIZABLE')])

```

Output

|   | level           | first read | writer     | writer wait (s) | second read | ค่าหลังจบ |
|---|-----------------|------------|------------|-----------------|-------------|-----------|
| 0 | READ COMMITTED  | 0          | committed  | 0.0             | 999         | 999       |
| 1 | REPEATABLE READ | 0          | committed  | 0.0             | 0           | 999       |
| 2 | SERIALIZABLE    | 0          | error 1205 | 3.0             | 0           | 0         |

ผลที่ได้ (reader อ่าน 2 ครั้ง ระหว่างนั้น writer เปลี่ยนค่าเป็น 999)

| Level           | อ่านครั้งที่ 2 | writer                      | อธิบาย                                                                                                           |
|-----------------|----------------|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| READ COMMITTED  | 999            | commit ได้ทันที             | ทุก SELECT สร้าง snapshot ใหม่ จึงเห็นค่าที่เพิ่ง commit (non-repeatable read)                                   |
| REPEATABLE READ | 0              | commit ได้ทันที             | snapshot ถูกสร้างตอน SELECT ครั้งแรก อ่านครั้งที่ 2 จึงได้ค่าเดิม (MVCC) และไม่ block writer                     |
| SERIALIZABLE    | 0              | รอ 3 วินาที แล้ว error 1205 | เมื่อปิด autocommit, SELECT ธรรมดาถูกเปลี่ยนเป็น locking read (shared lock) writer จึงต้องรอจน lock wait timeout |

ทั้ง RR และ SERIALIZABLE ให้ผลอ่านครั้งที่ 2 เหมือนกัน แต่ต่างกันที่กลไก RR ใช้ snapshot (writer ทำงานต่อได้) ส่วน SERIALIZABLE ใช้ lock (writer ถูก block) นี่คือการ trade-off ระหว่าง isolation กับ concurrency และเป็นเหตุผลที่ Lab ต้องตั้ง `innodb_lock_wait_timeout` ต่ำ ๆ ไม่อย่างนั้นโปรแกรมจะดูเหมือนค้างไป 50 วินาที (ค่า default)

## D. Deadlock

ใช้ผลจาก Lab 4 (สร้าง deadlock, ตรวจสอบ `SHOW ENGINE INNODB STATUS`, retry และแก้ด้วย lock ordering)

ส่วนนี้ใช้ผลจาก Lab 4 ในส่วนที่ 1 ซึ่งครอบคลุมทุกข้อที่โจทย์ต้องการ:

- **สร้าง deadlock:** UPDATE 2 แถวสลับลำดับกันใน 2 thread ได้ error 1213 (ไม่ได้ใช้ LOCK TABLES)
- **ตรวจสอบ InnoDB status:** อ่านส่วน LATEST DETECTED DEADLOCK เห็น lock ที่แต่ละฝั่งถือ/รอ และ transaction ที่ถูกเลือกเป็น victim
- **ทำ retry:** `run_with_retry()` ใช้ exponential backoff + jitter จนทั้งสอง transaction commit ได้และผลรวมข้อมูลถูกต้อง
- **ป้องกัน:** ล็อกแถวตามลำดับเดียวกัน ลด deadlock จาก 10/40 เหลือ 0/40



## E. Redo / Undo

```
query("""SELECT @@innodb_redo_log_capacity DIV 1048576 AS redo_capacity_MB,
              @@innodb_log_buffer_size DIV 1048576 AS log_buffer_MB,
              @@innodb_flush_log_at_trx_commit AS flush_at_commit,
              @@sync_binlog AS sync_binlog""")
```

Output

|   | redo_capacity_MB | log_buffer_MB | flush_at_commit | sync_binlog |
|---|------------------|---------------|-----------------|-------------|
| 0 | 100              | 16            | 1               | 1           |

เครื่องที่ทดลองตั้ง redo log capacity 100 MB, log buffer 16 MB และ flush redo ทุก commit (=1) ซึ่งเป็นค่าทำให้ durability เต็มที่

```
def get_redo_bytes():
    return global_status('Innodb_os_log_written')

reset_data()
measure = []

# 1) INSERT 1000 แถวแล้ว COMMIT
before = get_redo_bytes()
conn = get_connection(); cur = conn.cursor()
try:
    for i in range(1000):
        cur.execute("INSERT INTO posts(post_id, author_id, content, share_count) VALUES (%s, 'U001', %s, 0)",
                    (f'BULK_{i:04d}', f'Bulk insert test {i}'))
    conn.commit()
except Error:
    conn.rollback(); raise
finally:
    cur.close(); conn.close()
measure.append({'workload': 'INSERT 1000 แถว + COMMIT', 'redo_bytes': get_redo_bytes() - before})

# 2) UPDATE 1000 แถวแล้ว ROLLBACK
before = get_redo_bytes()
conn = get_connection(); cur = conn.cursor()
try:
    cur.execute("UPDATE posts SET content = CONCAT(content, ' (edited)') WHERE post_id LIKE 'BULK_%'")
    changed = cur.rowcount
    conn.rollback()
finally:
    cur.close(); conn.close()
measure.append({'workload': f'UPDATE {changed} แถว + ROLLBACK', 'redo_bytes': get_redo_bytes() - before})

# 3) อ่านอย่างเดียว
before = get_redo_bytes()
query("SELECT COUNT(*) FROM posts WHERE post_id LIKE 'BULK_%'")
measure.append({'workload': 'SELECT อย่างเดียว', 'redo_bytes': get_redo_bytes() - before})

edited = query("SELECT COUNT(*) AS n FROM posts WHERE content LIKE '%(edited)')").n[0]
print('แถวที่ยังมีคำว่า (edited) หลัง rollback =', edited)
pd.DataFrame(measure)
```

Output

แถวที่ยังมีคำว่า (edited) หลัง rollback = 0

|   | workload                   | redo_bytes |
|---|----------------------------|------------|
| 0 | INSERT 1000 แถว + COMMIT   | 288768     |
| 1 | UPDATE 1000 แถว + ROLLBACK | 419328     |
| 2 | SELECT อย่างเดียว          | 0          |

วัด redo bytes ก่อน/หลัง workload (Innodb\_os\_log\_written)

| Workload                    | Redo ที่เกิด  |
|-----------------------------|---------------|
| INSERT 1,000 แถว + COMMIT   | ประมาณ 289 KB |
| UPDATE 1,000 แถว + ROLLBACK | ประมาณ 419 KB |
| SELECT อย่างเดียว           | 0 byte        |

- การ **อ่านอย่างเดียวไม่สร้าง redo** เพราะไม่มี page ไหนเปลี่ยน
- น่าสนใจว่า **UPDATE แล้ว ROLLBACK สร้าง redo มากกว่า INSERT ที่ commit** ทั้งที่สุดท้ายข้อมูลไม่เปลี่ยนเลย (ไม่มีแถวไหนเหลือคำว่า "(edited)") เพราะตอน UPDATE InnoDB ต้องเขียน undo record เก็บค่าเก่า (การเขียน undo page ก็ถูกบันทึกลง redo ด้วย) และตอน rollback ก็เอา undo มาแก้ page กลับ ซึ่งเป็นการเปลี่ยน page อีกรอบ จึงต้องมี redo อีกเช่นกัน
- ค่า counter นี้เป็นค่ารวมของทั้ง server แต่เครื่องที่ใช้ไม่มี workload อื่น (SELECT ได้ 0 byte) จึงพอเชื่อได้ว่าเป็นของ workload ที่วัดจริง

### บทบาทของ redo กับ undo

- **Redo log:** บันทึกว่า page ถูกเปลี่ยนอย่างไร ใช้ตอน crash recovery เพื่อทำซ้ำการเปลี่ยนแปลงที่ commit แล้วแต่ยังไม่ได้เขียน data page ลง disk (Durability) ทำให้ตอน commit ไม่ต้องเขียน data page ทั้งหมด แค่ flush redo ก็พอ
- **Undo log:** เก็บค่าเก่าของแถว ใช้ rollback transaction ที่ไม่สำเร็จ (Atomicity) และใช้สร้าง versionเก่าให้ consistent read ของ MVCC (Isolation)

```
# Undo + MVCC: reader เปิด snapshot ไว้ก่อน แล้ว writer แก้และ commit
reset_data()
r_conn, w_conn = get_connection(), get_connection()
rc, wc = r_conn.cursor(), w_conn.cursor()
try:
    rc.execute("START TRANSACTION WITH CONSISTENT SNAPSHOT")
    rc.execute("SELECT content FROM posts WHERE post_id='P003'"); print('reader (ก่อน)      :', rc.fetchone()[0])

    wc.execute("UPDATE posts SET content='[แก้ไข] Government announces new policy' WHERE post_id='P003'")
    w_conn.commit(); print('writer commit แล้ว')

    rc.execute("SELECT content FROM posts WHERE post_id='P003'"); print('reader (snapshot เดิม):',
rc.fetchone()[0])
    r_conn.commit()
    rc.execute("SELECT content FROM posts WHERE post_id='P003'"); print('reader (transaction ใหม่):',
rc.fetchone()[0])
    r_conn.commit()
finally:
    rc.close(); wc.close(); r_conn.close(); w_conn.close()

reset_data() # ล้างแถว BULK_ และคืนข้อมูลตั้งต้น
```

### Output

```
reader (ก่อน)      : Government announces new policy
writer commit แล้ว
reader (snapshot เดิม): Government announces new policy
reader (transaction ใหม่): [แก้ไข] Government announces new policy
```

**หลักฐานของ undo ใน MVCC:** reader เปิด snapshot ไว้ก่อน แล้ว writer แก้ content ของ P003 และ commit ไปแล้ว แต่ reader ใน transaction เดิม ยังอ่านได้ข้อความเก่า InnoDB สร้าง versionเก่าของแถวนี้ขึ้นมาจาก undo log ให้ตรงกับ snapshot ของ reader พอเริ่ม transaction ใหม่ถึงจะเห็นค่าใหม่ reader ไม่ต้องรอ lock ของ writer เลย ซึ่งเป็นวิธีที่ MVCC ช่วยลด blocking (คำถามอภิปรายข้อ 3)

## F. สรุปเชิงวิเคราะห์

จากการทดลองทั้งหมด ผมสรุปความเชื่อมโยงของแต่ละกลไกได้ดังนี้

### 1. ACID ไม่ได้มาจาก COMMIT คำสั่งเดียว แต่ละข้อมาจากกลไกต่างกัน

| คุณสมบัติ   | กลไกที่ทำให้เกิด                                                             | เห็นจาก Lab  |
|-------------|------------------------------------------------------------------------------|--------------|
| Atomicity   | undo log + application ต้อง ROLLBACK เองเมื่อ error                          | งาน A, Lab 2 |
| Consistency | constraint (PK, UNIQUE, FK, CHECK) + business rule ในโค้ด เช่น ตรวจ rowcount | งาน A, Lab 2 |
| Isolation   | MVCC (snapshot จาก undo) + locking                                           | งาน B, C, E3 |
| Durability  | redo log ที่ flush ตอน commit (flush_at_trx_commit = 1)                      | Lab 3, งาน E |

**2. Locking กับ MVCC ทำงานคู่กัน** SELECT ธรรมดาใช้ snapshot จึงไม่ block และไม่ถูก block (งาน C ระดับ RR, E3) แต่ snapshot ไม่ได้ป้องกัน lost update (งาน B ได้ค่าหายเฉลี่ย 7.2 จาก 10) งานที่อ่านแล้วเขียนกลับจึงต้องใช้ atomic UPDATE หรือ locking read

**3. Isolation สูงขึ้นแลกกับ concurrency ที่ลดลง** READ COMMITTED เกิด non-repeatable read, REPEATABLE READ แก้ได้ด้วย snapshot โดยไม่ block ใคร ส่วน SERIALIZABLE ใช้ shared lock จน writer รอจน timeout ส่วน READ UNCOMMITTED อ่านค่าที่ถูก rollback ได้ (Lab 1) ไม่ควรใช้

**4. ใช้ lock เมื่อไหร่ต้องเตรียมรับ deadlock** deadlock เกิดได้แม้โค้ดจะถูกต้อง (Lab 2 แบบ naive, Lab 4) InnoDB ตรวจเจอและ rollback victim ให้ หน้าที่ของ application คือ retry ทั้ง transaction และออกแบบลำดับการล็อกให้เหมือนกันเพื่อลดโอกาสเกิด

**5. Recovery ขึ้นกับ redo และ undo** redo ทำให้ข้อมูลที่ commit แล้วไม่หายแม้เครื่องดับ ส่วน undo ใช้ย้อน transaction ที่ยังไม่ commit แต่การ commit ทุกครั้งต้อง fsync ทำให้ autocommit ที่ละแถวช้ากว่าประมาณ 3.8 เท่า (Lab 3) และแม้แต่ rollback ก็สร้าง redo (งาน E) การออกแบบขนาด transaction จึงต้องหาจุดพอดีระหว่างความเร็ว ระยะเวลาถือ lock และต้นทุน recovery

**นำไปใช้กับ Fake News Mini Project:** ตัวนับ (share\_count, report\_count) ใช้ atomic UPDATE, การ report/flag ทำใน transaction เดียวที่ล็อกแถวโพสต์ก่อน, ทุก transaction มี rollback ใน exception path และ retry เมื่อเจอ 1213/1205 และใช้ isolation ค่า default (REPEATABLE READ) ไม่ใช่ READ UNCOMMITTED